CareerLink Project Report

Author:
Bhushan Dattatray Sonawane
Student Email : 23f2003210@ds.study.iitm.ac.in
I am A Diploma Level Student of BS Degree in Data Science And Application At IIT Madras.

Project Details & Approach :

Problem Statement:
Institutes require efficient systems to manage campus recruitment activities involving companies and students. The objective is to build a modern, scalable Placement Portal Application (PPA) that digitizes and streamlines the process of company approvals, placement drives, student applications, and overall status tracking

Approach & Architecture:
The project was designed with a separated frontend and backend architecture to ensure scalability and a clear separation of concerns:

- Backend (Flask): Designed as a RESTful API using Flask alongside Flask-RESTful. The application heavily utilizes token-based authentication (via Flask-Security-Too) to manage three distinct roles: Admin, Student, and Company. Database interactions are securely abstracted using Flask-SQLAlchemy object-relational mapping (ORM).

- Frontend (VueJS 3): Built using modern Vue 3 Composition API to provide a reactive, Single Page Application (SPA) experience. It handles client-side routing, dynamic forms, and seamless dashboard transitions for all three user roles.

- Asynchronous Processing: Heavy and scheduled tasks (like generating comprehensive CSV data exports and batch email reminders) were offloaded from the main web server to a Celery task queue, utilizing Redis as the message broker.

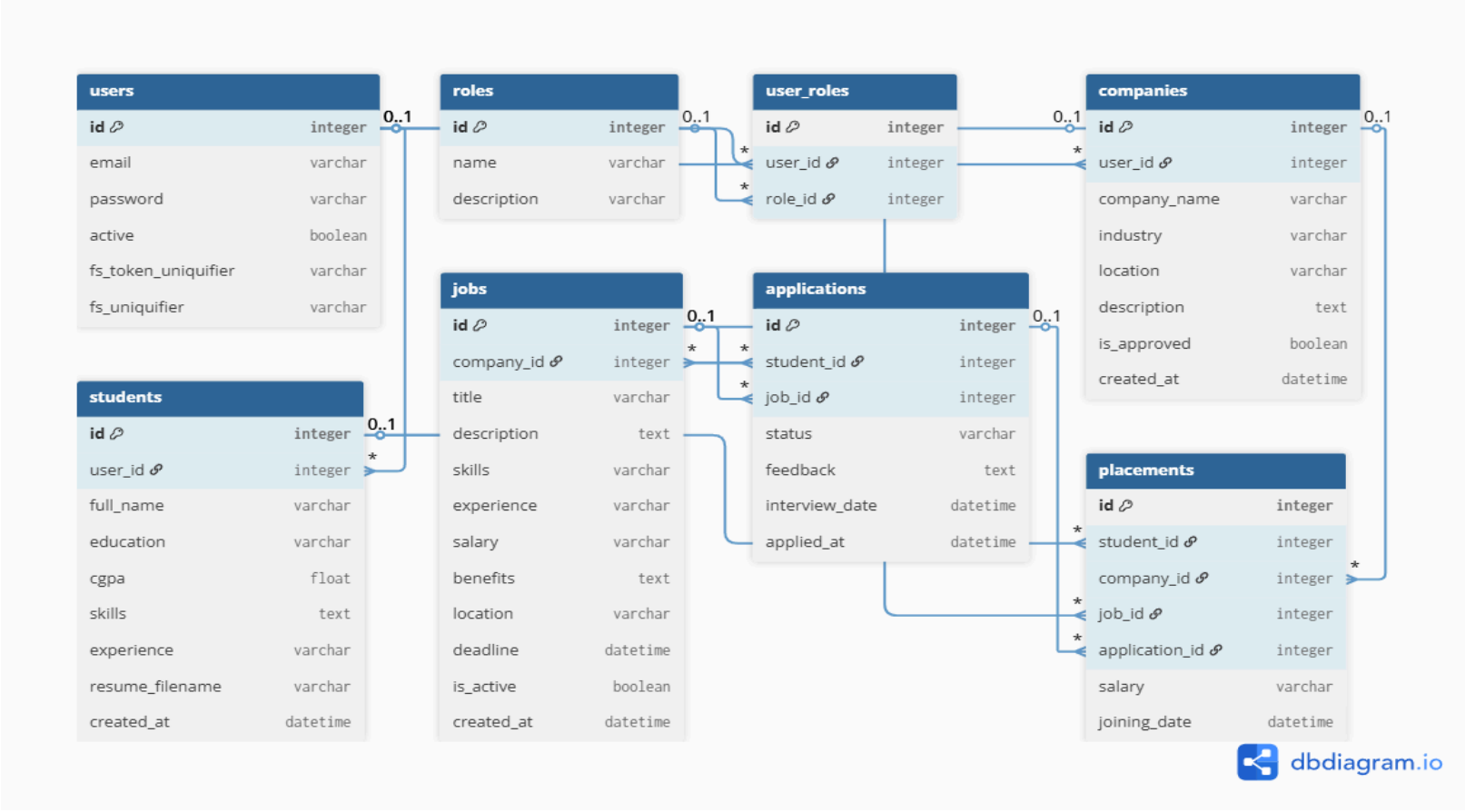
- Performance Optimization: Implemented caching strategies on heavily accessed API endpoints (such as job listings and dashboard statistics) using Redis and Flask-Caching, dramatically improving server response times.

Frameworks and Libraries Used :

The application strictly adhered to the mandated technology stack:

- Backend API: Python 3, Flask, Flask-RESTful, Flask-CORS
- Database & ORM: SQLite 3 (programmatically initialized), Flask-SQLAlchemy
- Authentication & Security: Flask-Security-Too (Token-based Auth)
- Frontend UI: Vue.js 3 (Composition API), Vite (Bundler), Vue Router
- Styling: Native CSS and Bootstrap (for rapid layout consistency)
- Task Queue & Scheduling: Celery, Celery Beat
- Broker & Caching: Redis, Flask-Caching
- PDF/Report Generation: FPDF, CSV Standard Library

Entity-Relationship (ER) Diagram :



The database schema revolves around tightly linked core entities:

- User: Base authentication table (id, email, password, active).
- Role & RolesUsers: Many-to-Many mapping handling Admin, Student, and Company role assignment
- Student: Links to `User` (1-to-1). Stores academic profile (full_name, cgpa, education).
- Company: Links to `User` (1-to-1). Stores corporate details (company_name, industry, location, is_approved).
- Job (Placement Drive): Links to `Company` (Many-to-1). Stores drive metadata (title, description, skills, salary, deadline, is_active)
- Application: Junction between `Student` and `Job` (Many-to-1 to Both). Tracks the lifecycle tracking of an application (status: Applied, Shortlisted, Selected, Rejected) and application timestamp.
- Placement: Historical tracking table explicitly linking hired Students to Companies and Jobs.

API Resource Endpoints:

The RESTful API is prefixed by `/api` and is fully secured via `Authentication-Token` headers. Key endpoints include:

- Authentication:
    - POST /register: Register a new Student or Company.
    - POST /login: Authenticates user and returns an auth token.
- Student Endpoints:
    - GET /student/profile, PUT /student/profile: Manage student details.
    - GET /student/jobs: Fetches cached active placement drives.
    - GET /student/applications POST /student/applications: Apply to and view job status.
- Company Endpoints:
    - GET /company/profile, PUT /company/profile: Manage company specifics.
    - GET /company/jobs`, `POST /company/jobs`, `PUT /company/jobs/`: Manage placement drives.
    - GET /company/jobs//applications`, `PUT /company/applications/`: Process applicants.
- Admin Endpoints:
    - GET /admin/dashboard`: Cached top-level system statistics.
    - GET /admin/companies`, `POST /admin/companies`: Browse and approve/revoke companies.

    - GET /admin/students`, `POST /admin/students`: Browse and manage student access.
    - GET /admin/jobs`, `POST /admin/jobs`: Moderate placement drives globally.
- Export Tasks:
    - POST /export`: Submits a Celery batch job to generate application CSV data
    - GET /export/status/`: Polls background job status.
    - GET /export/download/`: Retrieves generated file blob.

AI/LLM Declaration:

I used ChatGPT (GPT-5) to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency. The extent of AI/LLM
usage is around 15–20%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, and integration were done manually.

Video Presentation Link:

CareerLink Demo.mp4